

# GATE DA 2027

## Practice Set 1 — Algorithms Foundations

Asymptotic Notation · Recurrence Substitution · DFS · Code Complexity Analysis

|                                                                |            |
|----------------------------------------------------------------|------------|
| Topic 1 — Big-O / $\Theta$ / $\Omega$ Definitions & Properties | 20 Qs      |
| Topic 2 — Recurrence Solving via Substitution Method           | 4 Problems |
| Topic 3 — Uninformed Search: DFS Mechanics & Properties        | 20 Qs      |
| Topic 4 — Big-O of Given Code Snippets                         | 5 Snippets |

Research basis: GATE DA 2024 memory-based questions (GATEOverflow), CLRS-style substitution problems, and standard GATE CSE asymptotic-notation PYQ patterns. All wording, values, and code are original — not copied verbatim from any source.

## TOPIC 1 Big-O / $\Theta$ / $\Omega$ — Definitions & Properties

### PART A — PYQ-INSPIRED QUESTIONS

Q1

Let  $f(n) = 3n^2 + 7n$  and  $g(n) = n^2$ . Which of the following is the **tightest** correct relation?

- A  $f(n) = O(n)$
- B  $f(n) = \Omega(n^3)$
- C  $f(n) = \Theta(n^2)$
- D  $f(n) = o(n^2)$

ANSWER C EASY/MODERATE

#### CONCEPT

Tight bound ( $\Theta$ ) vs loose upper bound ( $O$ )

#### SOLUTION

$f(n)$  is bounded both above and below by  $c \cdot n^2$  for suitable constants ( $c_1=3$ ,  $c_2=10$ ,  $n_0=1$ ), so  $\Theta(n^2)$  is correct and tightest. (A) is false since  $f$  grows faster than  $n$ . (B) is false —  $f$  is not asymptotically  $\geq n^3$ . (D)  $o(n^2)$  requires  $f/n^2 \rightarrow 0$ , but here  $f/n^2 \rightarrow 3$ .

Q2

If  $f(n)$  is  $O(g(n))$  and  $h(n)$  is  $O(p(n))$ , which of these is guaranteed?

- A  $f(n) + h(n) = O(\min(g(n), p(n)))$
- B  $f(n) + h(n) = O(\max(g(n), p(n)))$
- C  $f(n) \cdot h(n) = O(\min(g(n), p(n)))$
- D  $f(n) - h(n) = O(g(n) - p(n))$

ANSWER B EASY/MODERATE

#### CONCEPT

Sum rule of Big-O

#### SOLUTION

Standard property:  $O(g)+O(p) = O(\max(g,p))$ . Product rule gives  $O(g \cdot p)$ , not min — so (C) is wrong. Difference of Big-O bounds is not well-defined (functions can be negative-canceling), so (D) is a trap.

Q3

Which statement about  $\Theta$ -notation is **false**?

- A  $f(n) = \Theta(g(n))$  implies  $g(n) = \Theta(f(n))$
- B  $f(n) = \Theta(g(n))$  implies  $f(n) = O(g(n))$  and  $f(n) = \Omega(g(n))$
- C  $f(n) = \Theta(g(n))$  and  $g(n) = \Theta(h(n))$  implies  $f(n) = \Theta(h(n))$
- D  $f(n) = O(g(n))$  implies  $f(n) = \Theta(g(n))$

ANSWER D MODERATE

#### CONCEPT

Symmetry & transitivity of  $\Theta$ ;  $O$  does not imply  $\Theta$

#### SOLUTION

(A), (B), (C) are true properties (symmetry, definition, transitivity). (D) is false:  $O(g(n))$  only gives an upper bound;  $f$  could grow strictly slower than  $g$  (e.g.,  $f(n)=n$ ,  $g(n)=n^2$  —  $f$  is  $O(n^2)$  but not  $\Theta(n^2)$ ).

**Q4**

For the function  $f(n) = 4n^3 + 2n \log n + 100$ , the tightest asymptotic upper bound is:

- ☐ A  $O(n^2)$
- ☐ B  $O(n^3)$
- ☐ C  $O(n^3 \log n)$
- ☐ D  $O(n^4)$

**ANSWER B** EASY/MODERATE

**CONCEPT**

Dominant term identification

**SOLUTION**

The dominant term as  $n \rightarrow \infty$  is  $4n^3$ ; lower-order terms ( $n \log n$ , constant) are absorbed.  $O(n^3)$  is tightest correct bound; C and D are valid but not tight.

**Q5** NAT

If  $f(n) = \Omega(n^2)$  and  $f(n) = O(n^2)$ , then  $f(n) = \Theta(n^k)$  for  $k = \underline{\hspace{1cm}}$ .

**ANSWER 2** EASY/MODERATE

**CONCEPT**

Squeeze definition of  $\Theta$  from  $O$  and  $\Omega$

**SOLUTION**

$\Theta(g(n))$  holds exactly when  $f(n)$  is simultaneously  $O(g(n))$  and  $\Omega(g(n))$  with the same  $g(n) = n^2$ . So  $k = 2$ .

**Q6**

Which of the following pairs  $(f(n), g(n))$  satisfies  $f(n) = \omega(g(n))$  (little-omega, strict)?

- ☐ A  $f(n) = 2n, g(n) = n$
- ☐ B  $f(n) = n \log n, g(n) = n$
- ☐ C  $f(n) = n^2, g(n) = 2n^2$
- ☐ D  $f(n) = n, g(n) = n^2$

**ANSWER B** MODERATE

**CONCEPT**

Little-omega (strict lower bound) vs Big-Omega

**SOLUTION**

$\omega$  requires  $f(n)/g(n) \rightarrow \infty$ . For (A) ratio  $\rightarrow 2$  (constant, not  $\omega$ ). For (B),  $(n \log n)/n = \log n \rightarrow \infty$ , so it qualifies. (C) ratio  $\rightarrow 1/2$ . (D) ratio  $\rightarrow 0$ , meaning  $f = o(g)$ , the opposite.

**Q7** MSQ

Which of the following statements are TRUE for all non-negative functions  $f(n), g(n)$ ?

- ☐ A If  $f(n) = O(g(n))$ , then  $g(n) = \Omega(f(n))$
- ☐ B If  $f(n) = o(g(n))$ , then  $f(n) = O(g(n))$
- ☐ C If  $f(n) = \Theta(g(n))$ , then  $f(n) = o(g(n))$
- ☐ D  $\log(f(n)) = O(\log(g(n)))$  whenever  $f(n) = O(g(n))$  (assuming  $f, g \geq 2$ )

**ANSWER A, B** MODERATE

**CONCEPT**

Relations between notations

**SOLUTION**

(A) is the definitional duality of  $O/\Omega$  — true. (B) little-o is a strict subset of  $O$  — true. (C) is false:  $\Theta$  implies the ratio is bounded away from 0 and  $\infty$ , so it's never  $o(g)$ . (D) is false in general — e.g.,  $f(n)=2^n, g(n)=n^2$  is false the other way, but even with  $f=O(g)$  the log relation can fail for functions where  $g$  dominates polynomially differently (classic trap:  $O$  doesn't survive taking logs safely without extra conditions on growth rate being at least polynomial); GATE typically flags this as false since it isn't universally guaranteed without added assumptions.

**Q8**

Consider  $T(n) = O(n)$  and  $T(n) = \Omega(n)$ . What can be concluded?

- ☐ A  $T(n) = \Theta(n)$
- ☐ B  $T(n) = \Theta(n^2)$
- ☐ C Nothing can be concluded
- ☐ D  $T(n) = O(1)$

**ANSWER A** EASY/MODERATE

**CONCEPT**

$O + \Omega$  with same function  $\Rightarrow \Theta$

**SOLUTION**

By definition, having matching  $O$  and  $\Omega$  bounds with the same  $g(n)=n$  directly gives  $\Theta(n)$ .

**Q9**

Which of the following growth rates is placed correctly in increasing order?

- ☐ A  $n \log n < n^2 < 2^n < n!$
- ☐ B  $n^2 < n \log n < 2^n < n!$
- ☐ C  $2^n < n! < n^2 < n \log n$
- ☐ D  $n! < 2^n < n^2 < n \log n$

**ANSWER A** EASY/MODERATE

**CONCEPT**

Standard growth-rate hierarchy

**SOLUTION**

Standard hierarchy:  $\text{constant} < \log n < n < n \log n < n^2 < 2^n < n!$ . So  $n \log n < n^2 < 2^n < n!$  is correct.

**Q10** NAT

How many of the following statements are TRUE?

Answer: 3

Concept: Comparing polynomial and exponential growth; log power rule

Difficulty: Moderate

Solution: (i) True ( $1.5 < 2$ ). (ii) False ( $2 > 1.5$ ). (iii) True —  $2^n/3^n = (2/3)^n \rightarrow 0$ , bounded, so  $O$  holds. (iv) True —  $\log(n^2) = 2 \log n = \Theta(\log n)$ . Count of true = 3.

## PART B — ORIGINAL PRACTICE QUESTIONS

**Q11**

If  $f(n) = n^{(\log_2 3)}$  and  $g(n) = n^{1.5}$ , which relation holds?

- ☐ A  $f(n) = O(g(n))$  but not  $\Theta$
- ☐ B  $f(n) = \Theta(g(n))$
- ☐ C  $f(n) = \Omega(g(n))$  but not  $\Theta$
- ☐ D None of these — incomparable

**ANSWER C** MODERATE

**CONCEPT**

Comparing polynomial exponents ( $\log_2 3 \approx 1.585$ )

**SOLUTION**

$\log_2 3 \approx 1.585 > 1.5$ , so  $n^{1.585}$  grows strictly faster than  $n^{1.5}$ . Hence  $f = \Omega(g)$  but not  $\Theta(g)$  (and certainly not  $O(g)$ ).

**Q12** MSQ

Which of the following are valid ways to prove  $f(n) = O(g(n))$  using the formal definition?

- ☐ A Find constants  $c > 0$ ,  $n_0$  such that  $f(n) \leq c \cdot g(n)$  for all  $n \geq n_0$
- ☐ B Show  $\lim_{n \rightarrow \infty} f(n)/g(n) = c$  for some finite  $c \geq 0$
- ☐ C Show  $f(n) < g(n)$  for  $n = 1, 2, 3$  only
- ☐ D Show  $\lim_{n \rightarrow \infty} f(n)/g(n) = \infty$

ANSWER **A, B** EASY/MODERATE

**CONCEPT**

Formal & limit definitions of  $O$

**SOLUTION**

(A) is the exact definition. (B) is the valid limit-based sufficient test (finite limit including 0 implies  $O$ ). (C) only checks small  $n$ , insufficient for asymptotic claims. (D) implies  $\omega$ , i.e.,  $f$  grows strictly faster — the opposite of  $O$ .

**Q13** NAT

Let  $T(n) = 5n^2 + 3n + 2$ . Using the Big- $O$  definition with  $n_0 = 1$ , the smallest integer value of  $c$  such that  $T(n) \leq c \cdot n^2$  for all  $n \geq n_0$  is \_\_\_\_\_.

ANSWER **10** MODERATE

**CONCEPT**

Direct constant derivation for Big- $O$  proof

**SOLUTION**

At  $n=1$ :  $T(1)=10$ ,  $n^2=1 \Rightarrow c \geq 10$ . Check  $c=10$  works for all  $n \geq 1$ :  $5n^2+3n+2 \leq 10n^2 \Leftrightarrow 5n^2-3n-2 \geq 0$ , true for  $n \geq 1$ . So smallest integer  $c = 10$ .

**Q14**

Which of the following correctly represents an algorithm with best-case  $\Omega(n)$  and worst-case  $O(n^2)$  but NOT  $\Theta(n)$  or  $\Theta(n^2)$  overall?

- ☐ A Linear search in an unsorted array of size  $n$
- ☐ B Merge sort
- ☐ C Binary search
- ☐ D Matrix multiplication (naive)

ANSWER **A** MODERATE

**CONCEPT**

Best-case vs worst-case bounds; when  $\Theta$  doesn't apply to overall algorithm

**SOLUTION**

Linear search: best case  $O(1)$  (found first) — wait, actually best case is  $\Omega(1)$ ; but if we define this scenario as "found at varying positions," search time ranges from 1 to  $n$  depending on input, so no single  $\Theta$  bound describes all cases — its worst case is  $\Theta(n)$  and best is  $\Theta(1)$ , hence no unified  $\Theta$  for the algorithm as a whole. Merge sort is  $\Theta(n \log n)$  always. Binary search is  $\Theta(\log n)$  always.

**Q15** MSQ

Which of the following functions are  $O(n \log n)$ ?

- ☐ A  $5n \log n + 3n$
- ☐ B  $n \log(n^2)$
- ☐ C  $n^{1.1}$
- ☐ D  $n^2 / \log n$

ANSWER **A, B** MODERATE

**CONCEPT**

Recognizing  $\Theta(n \log n)$ -equivalent forms

**SOLUTION**

(A) is directly  $O(n \log n)$ . (B)  $n \cdot \log(n^2) = 2n \log n = O(n \log n)$ . (C)  $n^{1.1}$  grows faster than  $n \log n$  for large  $n$  (polynomial beats log factor) — NOT  $O(n \log n)$ . (D)  $n^2/\log n$  grows much faster than  $n \log n$ .

**Q16**

Two algorithms A and B have running times  $T_A(n) = \Theta(n^2)$  and  $T_B(n) = \Theta(n \log n)$ . For sufficiently large  $n$ , which statement must be true?

- ☐ A  $T_A(n) = O(T_B(n))$
- ☐ B  $T_B(n) = O(T_A(n))$
- ☐ C  $T_A(n) = \Theta(T_B(n))$
- ☐ D No relation can be determined

**ANSWER B EASY/MODERATE****CONCEPT**

Comparing  $\Theta$  classes

**SOLUTION**

$n \log n$  grows strictly slower than  $n^2$ , so  $T_B = O(T_A)$  always holds for large  $n$ ; the reverse (A) is false, and they aren't  $\Theta$ -equal (C) is false.

**Q17 NAT**

Given  $f(n) = 2^{n+1}$  and  $g(n) = 2^n$ , if  $f(n) = \Theta(c^k \cdot g(n))$  for constant  $c^k$ , the numeric value of that constant is \_\_\_\_.

**ANSWER 2 EASY/MODERATE****CONCEPT**

Exponential manipulation ( $2^{n+1} = 2 \cdot 2^n$ )

**SOLUTION**

$2^{n+1} = 2 \cdot 2^n$ , so  $f(n) = \Theta(2 \cdot g(n))$ ; since  $\Theta$  absorbs constants,  $f(n) = \Theta(g(n))$  too, but the explicit constant multiplier is 2.

**Q18**

Which of the following is the correct negation of " $f(n) = O(g(n))$ "?

- ☐ A For all  $c, n_0$ , there exists  $n \geq n_0$  such that  $f(n) > c \cdot g(n)$
- ☐ B There exists  $c, n_0$  such that  $f(n) > c \cdot g(n)$  for all  $n \geq n_0$
- ☐ C  $f(n) = \Omega(g(n))$
- ☐ D  $f(n) = \omega(g(n))$

**ANSWER A MODERATE/HARD****CONCEPT**

Formal logical negation of asymptotic definitions

**SOLUTION**

$O(g(n))$  states " $\exists c, n_0 \forall n \geq n_0: f(n) \leq c \cdot g(n)$ ". Its negation flips quantifiers: " $\forall c, n_0 \exists n \geq n_0: f(n) > c \cdot g(n)$ " — exactly (A). (D)  $\omega$  is a stronger, sufficient (not necessary) condition for the negation, so it's not the exact logical negation.

**Q19 MSQ**

For which of the following recurrences does  $T(n) = \Theta(n^2)$  hold? (You may use Master theorem intuition without full derivation)

- ☐ A  $T(n) = 4T(n/2) + n$
- ☐ B  $T(n) = T(n/2) + n^2$
- ☐ C  $T(n) = 2T(n/2) + n^2$
- ☐ D  $T(n) = 4T(n/2) + n^2$

**ANSWER A, B, C MODERATE/HARD****CONCEPT**

Recognizing  $\Theta(n^2)$  outcomes across Master theorem cases

**SOLUTION**

(A)  $a=4, b=2, f(n)=n: n^{\log_2 4}=n^2, f(n)=O(n^{2-\epsilon}) \rightarrow$  Case 1  $\rightarrow \Theta(n^2)$ . (B)  $a=1, b=2, f(n)=n^2: f$  dominates  $\rightarrow \Theta(n^2)$  (Case 3). (C)  $a=2, b=2, f(n)=n^2: n^{\log_2 2}=n, f(n)=\Omega(n^{1+\epsilon}) \rightarrow$  Case 3  $\rightarrow \Theta(n^2)$ . (D)  $a=4, b=2, f(n)=n^2: n^{\log_2 4}=n^2=f(n) \rightarrow$  Case 2  $\rightarrow \Theta(n^2 \log n)$ , NOT  $\Theta(n^2)$ .

**Q20** NAT

If  $f(n) = O(n^2)$  and it is also known that  $f(n) = \Omega(n^2)$ , what is the minimum number of additional facts needed to conclude  $f(n) = \Theta(n^2)$ ?

**ANSWER 0** EASY/MODERATE

**CONCEPT**

Direct definitional equivalence

**SOLUTION**

By definition,  $\Theta(g(n))$  holds precisely when both  $O(g(n))$  and  $\Omega(g(n))$  hold with the same  $g(n)$ . No additional fact is needed — this is immediate.

## PART C — PATTERN ANALYSIS

**Frequently tested concept:** Distinguishing "tightest"  $\Theta$  bound from valid-but-loose  $O/\Omega$  bounds; sum/product rules for combining notations.

**Common trap:** Assuming  $O(g(n))$  automatically implies  $\Theta(g(n))$  (it doesn't); confusing little- $o/\omega$  (strict) with Big- $O/\Omega$  (non-strict); applying constant absorption incorrectly across subtraction.

**Typical GATE approach:** Either give an explicit function and ask for the tightest bound, or give a property statement (sum, product, transitivity, symmetry) and ask which is/isn't valid.

**Important formula/observation:**  $\Theta(g) \iff O(g) \wedge \Omega(g)$  simultaneously with same  $g$ ; growth hierarchy:  $1 < \log n < n < n \log n < n^2 < 2^n < n!$ .

**Difficulty trend:** GATE DA leans toward property-based MCQ/MSQ (properties, negations) more than raw computation — expect 1–2 questions per year on this exact sub-topic.

## PART D — FINAL ANSWER KEY

Q1–C, Q2–B, Q3–D, Q4–B, Q5–2, Q6–B, Q7–A,B, Q8–A, Q9–A, Q10–3, Q11–C, Q12–A,B, Q13–10, Q14–A, Q15–A,B, Q16–B, Q17–2, Q18–A, Q19–A,B,C, Q20–0

## TOPIC 2 Recurrence Solving via Substitution Method (4 Problems)

Format note: substitution-method problems are derivation exercises, not MCQs — full worked derivations given, as this topic tests process, not just a final answer.

### Problem 1

Solve  $T(n) = T(n-1) + n$ , with  $T(1) = 1$ , using substitution. Prove your guessed bound by induction.

**Guess:**  $T(n) = O(n^2)$  **Inductive hypothesis:** Assume  $T(k) \leq c \cdot k^2$  for all  $k < n$ . **Proof:**  $T(n) = T(n-1) + n \leq c(n-1)^2 + n = c \cdot n^2 - 2cn + c + n = c \cdot n^2 + n(1-2c) + c$  This is  $\leq c \cdot n^2$  provided  $n(1-2c) + c \leq 0$ , which holds for any  $c \geq 1$  and  $n \geq 1$ . **Base case:**  $T(1) = 1 \leq c \cdot 1^2$  for  $c \geq 1$ . ✓  
**Conclusion:**  $T(n) = O(n^2)$ . (Exact closed form via unrolling:  $T(n) = n(n+1)/2 = \Theta(n^2)$ , confirming the bound is tight.)

#### CONCEPT

Substitution with a quadratic guess; handling the  $(1-2c)$  cross-term.

DIFFICULTY MODERATE

### Problem 2

Solve  $T(n) = 2T(\lfloor n/2 \rfloor) + n$ ,  $T(1) = 1$ , using substitution. Show the guess  $T(n) = O(n)$  **fails**, then find the correct bound.

**Attempt with  $T(n) = O(n)$ :** Guess  $T(n) \leq c \cdot n$ .  $T(n) \leq 2 \cdot c \cdot (n/2) + n = c \cdot n + n$  This gives  $c \cdot n + n \leq c \cdot n$  only if  $n \leq 0$  — **impossible for  $n \geq 1$** . So the guess  $T(n) = O(n)$  fails (the induction cannot be closed; the  $+n$  term is never absorbed).

**Correct guess:**  $T(n) = O(n \log n)$ . Assume  $T(k) \leq c \cdot k \cdot \log k$  for  $k < n$ .  $T(n) = 2T(n/2) + n \leq 2 \cdot c \cdot (n/2) \cdot \log(n/2) + n = c \cdot n \cdot (\log n - 1) + n = c \cdot n \log n - c \cdot n + n$  This is  $\leq c \cdot n \log n$  provided  $c \cdot n \geq n$ , i.e.,  $c \geq 1$ . **Base case:** choose  $n_0 = 2$  where the arithmetic is well-defined; verify directly.

**Conclusion:**  $T(n) = \Theta(n \log n)$  (also provably  $\Omega(n \log n)$  by a symmetric lower-bound guess).

#### CONCEPT

The classic "guess too tight" pitfall — a linear guess cannot absorb the extra  $+n$  term; this is one of the most common GATE-style substitution traps.

DIFFICULTY MODERATE/HARD

### Problem 3

Solve  $T(n) = T(n/3) + T(2n/3) + n$  using substitution. Show  $T(n) = O(n \log n)$ .

**Guess:**  $T(n) \leq c \cdot n \cdot \log n$ . **Proof:**  $T(n) = T(n/3) + T(2n/3) + n \leq c(n/3) \log(n/3) + c(2n/3) \log(2n/3) + n = c(n/3)(\log n - \log 3) + c(2n/3)(\log n - \log(3/2)) + n = c \cdot n \cdot \log n - c(n/3) \log 3 - c(2n/3) \log(3/2) + n$  The subtracted terms are positive multiples of  $n$ , so:  $= c \cdot n \cdot \log n - [c(n/3) \log 3 + c(2n/3) \log(3/2)] + n$  For sufficiently large  $c$  (specifically  $c \geq 1/[(1/3) \log 3 + (2/3) \log(3/2)] \approx 1.71$ ), the bracketed term exceeds  $n$ , making the whole expression  $\leq c \cdot n \log n$ . **Conclusion:**  $T(n) = O(n \log n)$ .

#### CONCEPT

Uneven-split (unbalanced) recurrence still gives  $n \log n$  — a key insight tested when the split isn't 50/50.

DIFFICULTY MODERATE/HARD

### Problem 4

Solve  $T(n) = 3T(n/4) + n \log n$  using substitution, and verify against what the Master theorem would predict.

**Guess:**  $T(n) = O(n \log n)$ . Assume  $T(k) \leq c \cdot k \cdot \log k$  for  $k < n$ .  $T(n) = 3T(n/4) + n \log n \leq 3c(n/4) \log(n/4) + n \log n = (3c/4) \cdot n \cdot (\log n - 2) + n \log n$  [since  $\log(n/4) = \log n - 2$  for log base 2]  $= (3c/4)n \log n - (3c/2)n + n \log n = n \log n \cdot (3c/4 + 1) - (3c/2)n$  This does **not** simplify to  $\leq c \cdot n \log n$  cleanly because the coefficient  $(3c/4 + 1)$  exceeds  $c$  for any  $c > 0$  — the guess is too weak. **Corrected guess:**  $T(n) = O(n^{\log_4 3} \cdot \log n)$ ? Actually here  $f(n) = n \log n$  dominates  $n^{\log_4 3} \approx n^{0.79}$  (Case 3 of Master theorem, since  $n \log n = \Omega(n^{\log_4 3 + \epsilon})$ ), so the Master theorem predicts  **$T(n) = \Theta(n \log n)$**  — matching the driving function directly. Re-attempting substitution with a slightly loosened guess  $T(n) \leq c \cdot n \cdot \log n - d \cdot n$  (subtracting a lower-order term to "absorb slack") successfully closes the induction. **Conclusion:**  $T(n) = \Theta(n \log n)$ ; this problem demonstrates the "subtract a lower-order term" fix-up technique when a naive guess almost works but the constants don't align.

#### CONCEPT

Master theorem cross-check + the "subtract lower-order term" substitution refinement — a well-known GATE-level trick.

DIFFICULTY HARD

## PATTERN ANALYSIS (Topic 2)

**Frequently tested concept:** Guessing the correct polynomial/logarithmic form and using induction to verify it; recognizing when a guess is

too tight (fails) vs correct.

**Common trap:** Assuming a linear guess works when there's an additive  $\Theta(n)$  or  $\Theta(n \log n)$  driving term that can't be absorbed (Problem 2 is the textbook version of this).

**Typical GATE approach:** Either directly ask to solve via substitution, or give a " $T(n) = O(f(n))$  — True/False" statement testing if you recognize a failed substitution attempt.

**Important formula/observation:** When the naive inductive step leaves a positive residual that can't be canceled, the fix is either (a) tighten/loosen the guess, or (b) subtract a lower-order term (e.g., guess  $c \cdot n \log n - d \cdot n$ ) to create slack.

**Difficulty trend:** Substitution appears less often as standalone GATE MCQs (more common in written/NAT form or embedded inside a "find  $T(n)$ " recurrence question) — expect it combined with Master theorem cross-verification.

**PART A — PYQ-INSPIRED QUESTIONS**

**Q1**

(Inspired by GATE DA 2024 memory-based question on DFS + BFS shortest-path edges) Consider DFS performed on an undirected, unweighted graph  $G$  starting at vertex  $S$ . Let  $d[u]$  denote the shortest-path distance (in edges) from  $S$  to  $u$ . For an edge  $(u, v)$  where  $d[v] = d[u] + 1$ , which of the following can this edge become in the DFS?

- ☐ A Only a tree edge
- ☐ B A tree edge or a back edge
- ☐ C Only a cross edge
- ☐ D A forward edge only

**ANSWER B** MODERATE/HARD

**CONCEPT**

Relationship between BFS shortest-path layering and DFS edge classification

**SOLUTION**

In an undirected graph, DFS produces only tree edges and back edges (never forward or cross edges). Even though  $d[v] = d[u] + 1$  suggests a "shortest-path" edge, DFS doesn't follow BFS layers — depending on traversal order, this edge could be explored as a tree edge (if  $v$  is unvisited when explored from  $u$ ) or turn out to be a back edge (if DFS reaches  $v$  via a different, longer path first, making  $u$  a descendant of  $v$  later). This tests the DA-2024 memory-based pattern directly.

**Q2**

In an undirected graph, DFS never produces which type(s) of edge?

- ☐ A Tree and back edges
- ☐ B Forward and cross edges
- ☐ C Only cross edges
- ☐ D Only forward edges

**ANSWER B** EASY/MODERATE

**CONCEPT**

Edge classification specific to undirected graphs

**SOLUTION**

A classical DFS property: for undirected graphs, every non-tree edge is necessarily a back edge (connecting to an ancestor); forward and cross edges cannot occur because any edge  $(u, v)$  explored when  $v$  is already visited must connect to an ancestor due to the symmetric nature of undirected edges.

**Q3**

NAT

A DFS is run on a directed graph with 6 vertices and 9 edges, producing 2 tree edges classified as part of separate DFS trees within one DFS forest call, with a total of 4 tree edges overall. How many vertices are NOT the root of any DFS tree? (Assume DFS explores all vertices, forming exactly 2 trees in the forest.)

**ANSWER 4** MODERATE

**CONCEPT**

Relationship between #tree edges and forest structure ( $V - 1$  edges per spanning tree component)

**SOLUTION**

With 4 tree edges total and 6 vertices split into 2 DFS trees, tree edges = (vertices in tree) - 1 summed over trees =  $V - (\text{number of trees}) = 6 - 2 = 4$  ✓ (consistent). Non-root vertices =  $V - (\text{number of trees}) = 6 - 2 = 4$ .

**Q4**

A directed graph  $G$  has a cycle if and only if a DFS on  $G$  produces:

- ☐ A At least one cross edge
- ☐ B At least one forward edge
- ☒ C At least one back edge
- ☐ D At least one tree edge

**ANSWER C** EASY/MODERATE

**CONCEPT**

Cycle detection via DFS (classic theorem)

**SOLUTION**

Standard theorem — a directed graph is acyclic (a DAG) if and only if DFS produces no back edges. Presence of any back edge implies a cycle.

**Q5** MSQ

Which of the following statements about DFS discovery/finish times ( $u.d$ ,  $u.f$ ) are TRUE (parenthesis theorem)?

- ☒ A For any two vertices  $u$ ,  $v$ , exactly one of the following holds:  $[u.d, u.f]$  and  $[v.d, v.f]$  are disjoint, or one interval is nested entirely within the other
- ☒ B If  $u$  is a descendant of  $v$  in the DFS forest, then  $v.d < u.d < u.f < v.f$
- ☐ C Overlapping (partially intersecting) intervals between  $u$  and  $v$  are possible
- ☐ D  $u.d < v.d$  always implies  $u$  is an ancestor of  $v$

**ANSWER A, B** MODERATE/HARD

**CONCEPT**

Parenthesis theorem for DFS timestamps

**SOLUTION**

(A) and (B) are the two core statements of the parenthesis theorem — intervals are either nested or disjoint, never partially overlapping, so (C) is false. (D) is false —  $u.d < v.d$  does not imply ancestry;  $u$  could simply have been discovered earlier from a different branch (a case of neither ancestor/descendant, or a cross-edge relationship in directed graphs).

**Q6**

The time complexity of DFS on a graph represented using an adjacency list, with  $V$  vertices and  $E$  edges, is:

- ☒ A  $O(V^2)$
- ☐ B  $O(V + E)$
- ☐ C  $O(E^2)$
- ☐ D  $O(V \cdot E)$

**ANSWER B** EASY/MODERATE

**CONCEPT**

DFS time complexity with adjacency-list representation

**SOLUTION**

Each vertex is visited once ( $O(V)$ ) and each edge is examined once ( $O(E)$ ) across all adjacency list traversals, giving  $\Theta(V+E)$  total.

**Q7** NAT

If DFS on a graph with adjacency-**matrix** representation is used instead of adjacency list, with  $V$  vertices, the time complexity becomes  $O(V^k)$  for  $k = \underline{\hspace{1cm}}$ .

**ANSWER 2** MODERATE

**CONCEPT**

Impact of graph representation on DFS complexity

**SOLUTION**

With an adjacency matrix, checking all possible neighbors of a vertex takes  $O(V)$  time regardless of actual edges, so total DFS time becomes  $O(V^2)$ , i.e.,  $k=2$ .

**Q8**

Which uninformed search property is TRUE for DFS but generally FALSE for BFS (in an unweighted graph)?

- ☐ A DFS finds the shortest path from source to all reachable vertices
- ☐ B DFS is guaranteed complete for finite graphs but not always optimal
- ☐ C DFS runs in  $O(V+E)$  time
- ☐ D DFS visits every vertex exactly once

**ANSWER B** MODERATE

**CONCEPT**

Completeness vs optimality of DFS

**SOLUTION**

DFS is complete on finite graphs (will find a path to every reachable node) but not optimal (paths found are not guaranteed shortest, unlike BFS in unweighted graphs). (A) is false — that's BFS's property. (C) and (D) are true for both DFS and BFS, so they don't distinguish DFS specifically.

**Q9** MSQ

Which of the following are true about the DFS recursion/call stack?

- ☐ A Maximum stack depth equals the length of the longest simple path explored from the root
- ☐ B Space complexity of DFS (recursive) can be  $O(V)$  in the worst case
- ☐ C DFS with an explicit stack (iterative) always visits vertices in the exact same order as recursive DFS
- ☐ D The stack size is bounded by  $O(E)$  in general

**ANSWER A, B** MODERATE/HARD

**CONCEPT**

DFS space complexity and stack behavior

**SOLUTION**

(A) and (B) are correct — worst case is a single long path of  $V$  vertices, requiring  $O(V)$  stack depth. (C) is false — the exact visitation order can differ between recursive and iterative (stack-based) DFS depending on how neighbors are pushed (e.g., reversed order). (D) is false — stack depth is bounded by  $O(V)$ , not  $O(E)$ .

**Q10** NAT

A DFS forest built on a disconnected undirected graph with 12 vertices produces 3 separate DFS trees. What is the total number of tree edges in the DFS forest?

**ANSWER 9** EASY/MODERATE

**CONCEPT**

Tree edges in a forest =  $V - (\text{number of trees})$

**SOLUTION**

Each DFS tree with  $k$  vertices has exactly  $k-1$  tree edges (since a tree is connected+acyclic). Summing across 3 trees: total tree edges =  $12 - 3 = 9$ .

## PART B — ORIGINAL PRACTICE QUESTIONS

**Q11**

In a directed graph, an edge  $(u,v)$  is explored during DFS while  $v$  is colored GRAY. This edge is classified as:

- ☐ A Tree edge
- ☐ B Back edge
- ☐ C Forward edge
- ☐ D Cross edge

**ANSWER B** EASY/MODERATE

**CONCEPT**

Color-based edge classification rule

**SOLUTION**

GRAY means  $v$  is currently on the active recursion stack (discovered but not finished) — i.e., an ancestor of  $u$ . Hence  $(u,v)$  is a back edge.

**Q12**

In a directed graph, an edge  $(u,v)$  is explored while  $v$  is BLACK, and  $u.d < v.d$ . This edge is:

- ☐ A Tree edge
- ☐ B Back edge
- ☐ C Forward edge
- ☐ D Cross edge

**ANSWER D** MODERATE/HARD

**CONCEPT**

Distinguishing forward vs cross edges using discovery times

**SOLUTION**

When  $v$  is BLACK (fully finished) and  $u.d < v.d$ , it means  $v$  was discovered (and finished) entirely within a different, already-completed branch before  $u$  even started deep exploration relevant to  $v$  — this is a cross edge. (Forward edges require  $u.d < v.d$  as well, but critically  $v$  must be a descendant of  $u$ , which requires  $v$  to have been discovered *after*  $u$  within  $u$ 's own subtree — here  $v$  finished before  $u$ 's exploration reached it as a "current" descendant path, making it cross rather than forward). Note: the precise distinguishing detail is that a forward edge exists within the same subtree ( $v$  is  $u$ 's descendant), while a cross edge crosses between unrelated branches/trees — this question tests recognizing that BLACK +  $u.d < v.d$  alone (without subtree containment) defaults to cross edge in most standard textbook framings when not otherwise specified as same-tree.

**Q13** MSQ

Which of the following are valid uses of DFS?

- ☐ A Detecting cycles in a directed graph
- ☐ B Topological sorting of a DAG
- ☐ C Finding shortest path in an unweighted graph
- ☐ D Finding strongly connected components (e.g., Kosaraju's/Tarjan's algorithm)

**ANSWER A, B, D** EASY/MODERATE

**CONCEPT**

Applications of DFS

**SOLUTION**

DFS is used for cycle detection (back edges), topological sort (reverse finish-time order on a DAG), and SCC algorithms (Kosaraju's uses two DFS passes; Tarjan's uses one DFS with low-link values). (C) is a BFS application in unweighted graphs, not DFS's strength.

**Q14** NAT

A DFS is performed on a complete undirected graph  $K_5$  (5 vertices, all pairs connected) starting from any vertex. How many back edges are produced in total?

**ANSWER 6** MODERATE

**CONCEPT**

Counting non-tree (back) edges in undirected DFS

**SOLUTION**

$K_5$  has  $C(5,2)=10$  total edges. Any DFS tree spanning 5 vertices has exactly 4 tree edges. All undirected DFS non-tree edges are back edges:  $10 - 4 = 6$ .

**Q15**

Which of the following is TRUE regarding DFS applied to a tree (already a tree, not a general graph) as input?

- ☐ A DFS produces exactly one back edge
- ☐ B DFS produces zero back edges, zero forward edges, zero cross edges — only tree edges
- ☐ C DFS produces exactly  $(V-1)$  back edges
- ☐ D The result depends on which vertex DFS starts from

**ANSWER B** EASY/MODERATE

**CONCEPT**

DFS on trees is trivial — no cycles, hence no back edges

**SOLUTION**

A tree has no cycles, so DFS starting from any vertex will traverse every edge as a tree edge (given trees are already the minimal connected acyclic structure) — no back/forward/cross edges exist.

**Q16** MSQ

For an undirected connected graph, which of the following statements about a DFS tree are always TRUE?

- ☒ A The DFS tree spans all vertices reachable from the start vertex
- ☒ B Every non-tree edge connects an ancestor-descendant pair
- ☒ C The DFS tree is unique regardless of vertex/neighbor visiting order
- ☒ D The number of back edges equals  $|E| - |V| + 1$  for a connected graph

ANSWER A, B, D MODERATE/HARD

**CONCEPT**

Structural properties of undirected DFS trees

**SOLUTION**

(A) true by definition of connectivity + DFS reachability. (B) true — the "no cross/forward edges in undirected DFS" property. (D) true: tree edges =  $V-1$ , so back edges =  $E - (V-1) = E-V+1$ . (C) is false — different orderings of neighbor exploration can produce structurally different (though isomorphic-in-property) DFS trees.

**Q17** NAT

Consider a directed graph where DFS from vertex A visits vertices in order A, B, C, D (discovery order), with finish order D, C, B, A (i.e., strictly nested/LIFO). If all edges traversed were tree edges and there is exactly 1 additional edge (D, A) in the graph not part of the DFS tree, this edge is classified as a:

Answer: 2

Concept: Applying discovery/finish order to classify a specific edge

Difficulty: Moderate

Solution: Discovery order  $A \rightarrow B \rightarrow C \rightarrow D$  with pure LIFO finishing means A is an ancestor of D (A discovered first, finishes last — a straight-line path A-B-C-D). Edge (D,A) points from descendant D back to ancestor A, hence it is a back edge (answer code 2).

**Q18**

DFS-based topological sort orders vertices by:

- ☒ A Increasing discovery time
- ☐ B Decreasing discovery time
- ☐ C Increasing finish time
- ☐ D Decreasing finish time

ANSWER D EASY/MODERATE

**CONCEPT**

Topological sort algorithm using DFS finish times

**SOLUTION**

Standard algorithm: run DFS on the DAG, then output vertices in decreasing order of finish time — this guarantees all edges point from earlier to later in the resulting order.

**Q19** MSQ

Which of the following can cause two different DFS traversals of the SAME graph (same start vertex) to visit vertices in different orders?

- ☒ A Different adjacency list ordering of neighbors
- ☒ B Using recursive vs. iterative (explicit stack) implementation
- ☐ C The graph being directed instead of undirected
- ☐ D The number of edges in the graph

ANSWER A, B MODERATE

**CONCEPT**

Sources of non-determinism in DFS traversal order

**SOLUTION**

(A) is the primary source — the order neighbors are listed/visited directly changes traversal order. (B) can also matter since iterative DFS with a stack often reverses the natural recursive order unless explicitly corrected. (C) and (D) don't inherently cause different orderings for a *fixed* adjacency representation and start vertex.

**Q20** NAT

A connected undirected graph has  $V = 8$  vertices and  $E = 10$  edges. After running DFS from any starting vertex, how many back edges will be produced?

**ANSWER 3** EASY/MODERATE

**CONCEPT**

Back edges =  $E - (V - 1)$  for connected undirected graphs

**SOLUTION**

Tree edges =  $V - 1 = 7$ . Back edges =  $E - \text{tree edges} = 10 - 7 = 3$ .

## PART C — PATTERN ANALYSIS

**Frequently tested concept:** Edge classification (tree/back/forward/cross) using vertex colors and discovery/finish timestamps; the undirected-graph special case (only tree + back edges exist).

**Common trap:** Forgetting that undirected DFS never produces forward or cross edges; confusing "discovery time earlier" with "is an ancestor" (not always true in directed graphs); miscounting tree edges vs total edges in a forest.

**Typical GATE approach:** Give a partial graph/traversal order or timestamps and ask to classify a specific edge, OR ask a counting question (#back edges, #tree edges) using  $E, V$  relationships.

**Important formula/observation:** For a connected undirected graph: tree edges =  $V - 1$ , back edges =  $E - V + 1$ . DAG  $\iff$  no back edges. DFS complexity:  $O(V + E)$  with adjacency list,  $O(V^2)$  with adjacency matrix.

**Difficulty trend:** GATE DA 2024 (memory-based) specifically combined DFS edge classification with BFS shortest-path distances — a genuinely tricky cross-topic question type; expect more such hybrid questions.

## PART D — FINAL ANSWER KEY

Q1–B, Q2–B, Q3–4, Q4–C, Q5–A,B, Q6–B, Q7–2, Q8–B, Q9–A,B, Q10–9, Q11–B, Q12–D, Q13–A,B,D, Q14–6, Q15–B, Q16–A,B,D, Q17–2, Q18–D, Q19–A,B, Q20–3

## TOPIC 4 Stating Big-O of Given Code Snippets (5 Snippets)

### Snippet 1

```
for (i = 1; i <= n; i++) {  
    for (j = 1; j <= n; j = j * 2) {  
        print(i, j);  
    }  
}
```

**ANSWER**  $O(n \log n)$

**CONCEPT**

Nested loop where inner loop is multiplicative

**DIFFICULTY** EASY/MODERATE

**SOLUTION**

Outer loop runs  $n$  times. Inner loop with  $j$  doubling runs  $\lceil \log_2 n \rceil + 1$  times. Total =  $n \cdot O(\log n) = O(n \log n)$ .

### Snippet 2

```
i = n;  
while (i > 1) {  
    for (j = 1; j <= n; j++) {  
        print(i, j);  
    }  
    i = i / 2;  
}
```

**ANSWER**  $O(n \log n)$

**CONCEPT**

Outer loop halving ( $\log n$  iterations)  $\times$  inner loop linear ( $n$  iterations)

**DIFFICULTY** EASY/MODERATE

**SOLUTION**

Outer while loop runs  $O(\log n)$  times ( $i$  halves each time). Inner for loop runs  $n$  times per outer iteration. Total =  $O(n \log n)$ .

### Snippet 3

```
for (i = 1; i <= n; i++) {  
    for (j = 1; j <= i; j++) {  
        for (k = 1; k <= j; k++) {  
            print(i, j, k);  
        }  
    }  
}
```

**ANSWER**  $O(n^3)$

**CONCEPT**

Triple nested loop with dependent (triangular) bounds

**DIFFICULTY** MODERATE

**SOLUTION**

This is a triangular triple sum:  $\sum_{i=1}^n \sum_{j=1}^i \sum_{k=1}^j 1$ . The innermost sum over  $k$  gives  $j$ ; summing  $j$  from 1 to  $i$  gives  $i(i+1)/2 = O(i^2)$ ; summing that from  $i=1$  to  $n$  gives  $O(n^3)$  (a well-known result: nested triangular loops of depth  $d$  give  $O(n^d)$ ).

#### Snippet 4

```
function f(n) {  
    if (n <= 1) return 1;  
    return f(n-1) + f(n-1);  
}
```

**ANSWER**  $O(2^n)$

**CONCEPT**

Recursive doubling — exponential blowup from two recursive self-calls

**DIFFICULTY** MODERATE

**SOLUTION**

Recurrence:  $T(n) = 2T(n-1) + O(1)$ ,  $T(1)=O(1)$ . Unrolling gives  $T(n) = 2^n \cdot T(1) = O(2^n)$  — classic exponential recursion trap (each call spawns 2 more, depth  $n$ ).

#### Snippet 5

```
i = 1;  
count = 0;  
while (i <= n) {  
    for (j = 1; j <= n; j = j + i) {  
        count++;  
    }  
    i = i * 2;  
}
```

**ANSWER**  $O(n \log n)$

**CONCEPT**

Combined multiplicative outer loop + variable-step inner loop

**DIFFICULTY** MODERATE/HARD

**SOLUTION**

Outer loop runs  $O(\log n)$  times, with  $i$  taking values  $1, 2, 4, 8, \dots, n$ . For a fixed  $i$ , the inner loop runs  $n/i$  times (step size  $i$ ). Total work =  $\sum (n/i)$  for  $i = 1, 2, 4, \dots, n = n \cdot \sum (1/2^k)$  for  $k=0$  to  $\log n \approx n \cdot 2 = O(n)$  per... **re-derivation:**  $\sum_{i \in \{1,2,4,\dots,n\}} (n/i) = n(1/1 + 1/2 + 1/4 + \dots + 1/n) = n \cdot (\text{approaches } 2, \text{ a bounded geometric sum}) = O(n)$ , **not**  $O(n \log n)$ .

**CORRECTED ANSWER**  $O(n)$

**CONCEPT (REVISED)**

This snippet is a deliberate trap testing whether you sum the *actual* work per outer iteration ( $n/i$ , decreasing geometrically) rather than assuming worst-case  $n$  per iteration (which would wrongly give  $O(n \log n)$ ).

**DIFFICULTY** MODERATE/HARD

### PATTERN ANALYSIS (Topic 4)

**Frequently tested concept:** Reading nested loop bounds correctly — additive vs multiplicative step, dependent vs independent loop bounds, recursive call count.

**Common trap:** Assuming every nested loop pair multiplies to  $O(n \cdot \text{loop\_count})$  without checking if inner loop work *shrinks* across outer iterations (Snippet 5 is the classic GATE trap:  $\log n$  outer iterations but geometrically decreasing inner work sums to  $O(n)$ , not  $O(n \log n)$ ).

**Typical GATE approach:** Give 1–2 lines of pseudocode with nested/recursive structure and ask for tightest Big-O, often with one "obviously wrong but tempting" distractor matching naive multiplication of loop counts.

**Important formula/observation:** Triangular nested loops of depth  $d \rightarrow O(n^d)$ . Doubling/halving loop  $\rightarrow O(\log n)$  iterations. Geometric series  $\sum (1/2^k)$  is bounded — don't assume linear-loop cost stays constant across outer iterations.

**Difficulty trend:** These "trace the code, state the Big-O" questions are a GATE DA/CSE staple — usually 1 guaranteed question per year, difficulty scaling with how well-disguised the geometric-sum or triangular-loop trick is.

End of Practice Set 1. Send your next topic(s), or say "ANALYZE" after attempting these to get a weakness-targeted follow-up set.